

Universidade Mogi das Cruzes

Gabriel Agustín Fernández Alves

RGM: 11231102284

**PROJETO CASA INTELIGENTE: Automação e Monitoramento em Tempo Real com
ESP32 e MQTT junto ao Node-Red**

Mogi das Cruzes

2026

Sumário

Sumário.....	1
1. Introdução:	2
2. AMBIENTE DE DESENVOLVIMENTO E CONFIGURAÇÃO:	2
3. ESQUEMA DE CONEXÃO E HARDWARE:	3
4. FUNCIONAMENTO DO SISTEMA:	4
5. CÓDIGO FONTE (MicroPython):	4
5.1 Codigo Wokwi (main.py):.....	4
5.2 Codigo Wokwi (upload_manual.py):.....	12
5.3 Planilha com data hora, temperatura, umidade:	16
5.3 Node-red (fluxo):	17
5.4 Node-red (Dashboard):	18
5.5 Node-red (Dashboard - localhost):.....	19
5.6 Wokwi (Desing):.....	20
5.7 HaiveMQ:.....	20
5.8 Apps Script para o envio de Email:	21
5.9 Email simulado nos testes para envio de dados diários:	23

1. Introdução:

Este documento apresenta o desenvolvimento de um sistema de Casa Inteligente, focado no monitoramento de segurança e controle de atuadores em tempo real. O objetivo principal é garantir a leitura de múltiplos sensores, o controle autônomo e remoto de dispositivos da residência, e a notificação imediata do proprietário em caso de sinistros, utilizando a Internet das Coisas (IoT).

2. AMBIENTE DE DESENVOLVIMENTO E CONFIGURAÇÃO:

O projeto foi desenvolvido combinando hardware simulado e servidores locais de comunicação:

- **Editor e Firmware:** A lógica da placa foi elaborada em código-fonte MicroPython, configurado e aplicado utilizando o editor **VS Code** (Visual Studio Code) em conjunto com a extensão e o simulador **Wokwi**, garantindo testes fiéis ao hardware real.
- **Comunicação (Broker):** Utilizou-se o protocolo MQTT através do servidor público **HiveMQ** (`broker.hivemq.com`), que atua como o "correio" entre a casa e o painel.
- **Processamento e Dashboard:** O ambiente **Node-RED** foi configurado para assinar os tópicos MQTT, processar a lógica de retaguarda (Backend) e hospedar a interface visual (Dashboard Web) para o usuário final.
- **Banco de Dados e Alertas:** Integração direta com a API do **Google Sheets** para registro de histórico e disparo de e-mails em caso de emergência.

3. ESQUEMA DE CONEXÃO E HARDWARE:

Componente	Função no Sistema	Pino no ESP32	Tipo de Sinal
------------	-------------------	---------------	---------------

DHT22	Sensor de Clima (Temperatura e Umidade)	GPIO 18	Digital (In)
MQ2	Simulador de Sensor de Gás/Fumaça	GPIO 34	Analógico (ADC)
PIR	Sensor de Presença (Movimento)	GPIO 27	Digital (In)
LDR	Sensor de Luminosidade (Luz)	GPIO 35	Analógico (ADC)
Display OLED	Interface visual física da casa (I2C)	GPIO 32 (SCL), 33 (SDA)	I2C
Buzzer	Sirene de Alarme	GPIO 5	PWM
Relé 1	Controle da Luz	GPIO 23	Digital (Out)
Relé 2	Controle da Cortina	GPIO 22	Digital (Out)
Relé 3	Controle do Ar Condicionado	GPIO 21	Digital (Out)
Relé 4	Controle da Janela	GPIO 17	Digital (Out)

4. FUNCIONAMENTO DO SISTEMA:

O sistema opera em ciclos contínuos de leitura e ação, dividido em três frentes principais:

1. **Segurança e Proteção Máxima (Prioridade):** A placa avalia constantemente perigos iminentes. Se o gás ultrapassar o nível seguro (2000) ou a temperatura indicar incêndio ($>45^{\circ}\text{C}$), o sistema automaticamente liga a sirene, abre as janelas para ventilação, desliga o ar condicionado (para não espalhar fumaça) e aciona o alerta vermelho no painel, furando a fila de envios para disparar um e-mail de emergência imediatamente via Google Sheets.
2. **Automação de Conforto (Ar Condicionado):** O sistema possui um modo automático nativo. Quando ativo, avalia a temperatura para ligar ou desligar o relé do motor de forma autônoma, buscando manter o ambiente entre 20°C e 24°C .
3. **Comunicação Bidirecional via MQTT:** A cada 3 segundos, a placa empacota o estado de todos os sensores e relés em formato JSON e envia para o tópico `gabriel/casa/sensores`. O Node-RED lê este pacote, atualiza a Dashboard e hospeda uma página web. Ao clicar em um botão no painel web, o Node-RED publica o comando (ex: `luz_toggle`) no tópico `gabriel/casa/comandos`, que é lido pela placa para atracar o relé correspondente.

5. CÓDIGO FONTE (MicroPython):

5.1 Codigo Wokwi (main.py):

```
import machine
from machine import Pin, ADC, PWM, SoftI2C
import time
import dht
import network
from umqtt.simple import MQTTClient
import json
import urequests
import ssd1306

# --- TELA OLED ---
i2c = SoftI2C(scl=Pin(32), sda=Pin(33))
tela = ssd1306.SSD1306_I2C(128, 64, i2c)

def mensagem_tela_cheia(linha1, linha2="", linha3=""):
    tela.fill(0)
    tela.text(linha1, 0, 10)
    tela.text(linha2, 0, 30)
    tela.text(linha3, 0, 50)
    tela.show()

mensagem_tela_cheia("Iniciando...", "Aguarde.")

# --- PINOS DOS SENTIDOS DA CASA ---
sensor_clima = dht.DHT22(Pin(18))
sensor_gas = ADC(Pin(34))
sensor_gas.atten(ADC.ATTN_11DB)
sensor_presenca = Pin(27, Pin.IN)
sensor_luz = ADC(Pin(35))
sensor_luz.atten(ADC.ATTN_11DB)

# --- PINOS DOS MÚSCULOS DA CASA ---
buzzer = PWM(Pin(5))
rele_luz = Pin(23, Pin.OUT)
rele_cortina = Pin(22, Pin.OUT)
```

```
rele_ar = Pin(21, Pin.OUT)
rele_janela = Pin(17, Pin.OUT)

# CONFIGURAÇÕES EXTERNAS E EMAIL
URL_PLANILHA =
'https://script.google.com/macros/s/AKfycbw9BJEqROgelaUtEUb5Mb1WVoPXY3Mt1mfrVjHDl172RtE6JUQZ61zIT-IFt4\_sLmxh/exec'

# --- CONFIGURAÇÕES DA CASA ---
alarme_ligado = False
modo_ar_auto = True
ar_desligado_manual = False
temperatura_alvo = 22
estado_janela_manual = 0
modo_atual = "PADRAO"

SERVIDOR_MENSAGENS = "broker.hivemq.com"
NOME_DA_PLACA = "casa_inteligente_gabriel"
CANAL_ENVIAR_DADOS = "gabriel/casa/sensores"
CANAL_RECEBER_COMANDOS = "gabriel/casa/comandos"

ultimo_aviso_enviado = 0
INTERVALO_AVISOS = 3000

tempo_ultimo_sheets = 0
soma_temperatura_dia = 0.0
soma_umidade_dia = 0.0
leituras_dia = 0
alerta_ativo = False

def conecta_wifi():
    print("Ligando antena Wi-Fi...")
    mensagem_tela_cheia("Conectando", "Wi-Fi...")
    wifi = network.WLAN(network.STA_IF)
    wifi.active(True)
    if wifi.isconnected(): return
    wifi.connect("Wokwi-GUEST", "")
    while not wifi.isconnected():
        print(".", end="")
        time.sleep(0.5)
    print("\nWi-fi Conectado!")
    mensagem_tela_cheia("Wi-Fi", "Conectado!")
    time.sleep(1)

# --- OUVINDO OS BOTÕES DO NODE-RED ---
def ouve_node_red(topico, message):
```

```

global alarme_ligado, modo_ar_auto, temperatura_alvo,
estado_janela_manual, ar_desligado_manual, modo_atual

comando = message.decode('utf-8')
if len(comando) > 15 and "_" not in comando:
    return

if comando == "alarme_on":
    alarme_ligado = True
    print("\n[ SEGURANÇA ] -> Alarme ATIVADO!")
elif comando == "alarme_off":
    alarme_ligado = False
    desliga_barulho()
    print("\n[ SEGURANÇA ] -> Alarme DESATIVADO.")
elif comando == "cinema":
    if rele_cortina.value() == 0:
        rele_cortina.value(1)
        rele_luz.value(0)
        modo_atual = "CINEMA"
        print("\n[ MODO CINEMA ] -> ATIVADO!")
    else:
        rele_cortina.value(0)
        rele_luz.value(1)
        modo_atual = "PADRAO"
        print("\n[ MODO CINEMA ] -> DESATIVADO!")
elif comando == "luz_toggle":
    rele_luz.value(not rele_luz.value())
elif comando == "cortina_toggle":
    rele_cortina.value(not rele_cortina.value())
elif comando == "janela_toggle":
    estado_janela_manual = 1 if estado_janela_manual == 0 else 0
elif comando == "ar_auto":
    modo_ar_auto = True
    ar_desligado_manual = False
elif comando == "ar_off":
    modo_ar_auto = False
    ar_desligado_manual = True
elif comando == "ar_mais":
    modo_ar_auto = False
    ar_desligado_manual = False
    temperatura_alvo += 1
elif comando == "ar_menos":
    modo_ar_auto = False
    ar_desligado_manual = False
    temperatura_alvo -= 1

def prepara_mensageiro():

```

```

carteiro = MQTTClient(NOME_DA_PLACA, SERVIDOR_MENSAGENS)
carteiro.set_callback(ouve_node_red)
carteiro.connect()
carteiro.subscribe(CANAL_RECEBER_COMANDOS)
return carteiro

def toca_sirene():
    buzzer.freq(1000)
    buzzer.duty(512)

def desliga_barulho():
    buzzer.duty(0)

def enviar_email_relatorio(media_t, media_u, qtd_Leituras):
    print("\n[ E-MAIL ] -> INICIANDO ENVIO DE E-MAIL COM MÉDIAS...")
    mensagem_tela_cheia("Enviando...", "Relatorio", "por E-mail")
    try:
        dados_email = {
            "tipo": "email",
            "media_t": f"{media_t:.1f}",
            "media_u": f"{media_u:.1f}",
            "qtd_leituras": qtd_Leituras
        }
        res_email = urequests.post(URL_PLANILHA, json=dados_email)
        res_email.close()
        print("[ E-MAIL ] -> >>> ENTREGUE COM SUCESSO! <<<")
        mensagem_tela_cheia("E-mail", "Enviado", "com Sucesso!")
        time.sleep(2)
    except Exception as e:
        print("[ E-MAIL ] -> Falha ao enviar:", e)
        mensagem_tela_cheia("Erro no", "E-mail!")
        time.sleep(2)

def atualiza_display_principal(status, motivo):
    tela.fill(0)
    tela.text("Status: " + status, 0, 0)
    tela.text(motivo, 0, 20)
    tela.text("Modo: " + modo_atual, 0, 45)
    tela.show()

desliga_barulho()
rele_luz.value(0); rele_cortina.value(0)
rele_ar.value(0); rele_janela.value(0)

conecta_wifi()
mensageiro = prepara_mensageiro()

```



```

print("=====")
print("      CASA INTELIGENTE RODANDO      ")
print("=====")

while True:
    try:
        agora = time.ticks_ms()
        mensageiro.check_msg()

        sensor_clima.measure()
        temperatura = sensor_clima.temperature()
        umidade = sensor_clima.humidity()
        nivel_gas = sensor_gas.read()
        movimento = sensor_presenca.value()

        # --- DETECTAR MOTIVOS DE SEGURANÇA ---
        aviso_seguranca = "Nenhum Alerta"
        motivo_tela = "Tudo OK"
        status_casa = "SEGURA"

        if nivel_gas > 2000:
            aviso_seguranca = "Alto Nivel de Gas Detectado!"
            motivo_tela = "Vazamento Gas!"
            status_casa = "PERIGO"
        elif temperatura > 45:
            aviso_seguranca = "Temperatura Critica / Incendio!"
            motivo_tela = "Fogo/Calor!"
            status_casa = "PERIGO"
        elif alarme_ligado and movimento == 1:
            aviso_seguranca = "Invasor Detectado no Ambiente!"
            motivo_tela = "INVASOR!"
            status_casa = "PERIGO"

        atualiza_display_principal(status_casa, motivo_tela)

        # --- VERIFICADOR IMEDIATO DE PERIGO (FURA FILA) ---
        if status_casa == "PERIGO":
            if not alerta_ativo:
                alerta_ativo = True
                print(f"\n[ ALERTA URGENTE ] -> {aviso_seguranca} ->
Disparando e-mail imediatamente!")
                if URL_PLANILHA != '':
                    try:
                        dados_urgentes = {
                            "tipo": "leitura",

```

```

        "temperatura": temperatura,
        "umidade": umidade,
        "gas": nivel_gas,
        "aviso_seguranca": aviso_seguranca
    }
    res_urgente = urequests.post(URL_PLANILHA,
json=dados_urgentes)
    res_urgente.close()
    print("[ ALERTA URGENTE ] -> Enviado com sucesso para
a nuvem!")

    except Exception as erro_envio:
        print("[ ALERTA URGENTE ] -> Erro ao tentar enviar:",
erro_envio)
    else:
        alerta_ativo = False

# --- PROTEÇÃO MÁXIMA REAL ---
if temperatura > 45 or nivel_gas > 2000:
    toca_sirene()
    rele_janela.value(1)
    rele_ar.value(0)
else:
    if not (alarme_ligado and movimento == 1):
        desliga_barulho()
        rele_janela.value(estado_janela_manual)

    if modo_ar_auto:
        if temperatura > 28:
            rele_ar.value(1)
        elif temperatura < 17:
            rele_ar.value(1)
        elif 20 <= temperatura <= 24:
            rele_ar.value(0)
    elif ar_desligado_manual:
        rele_ar.value(0)
    else:
        if temperatura >= (temperatura_alvo + 1) or temperatura <=
(temperatura_alvo - 1):
            rele_ar.value(1)
        else:
            rele_ar.value(0)

    if alarme_ligado and movimento == 1:
        toca_sirene()

# --- ENVIO DE ROTINA PARA A PLANILHA DO GOOGLE (A CADA 20 SEGUNDOS) -
--

```

```

        if time.time_diff(agora, tempo_ultimo_sheets) > 20000:
            tempo_ultimo_sheets = agora

        print(f"\n[ PLANILHA ] -> Enviando leitura para o Google Sheets...
(Lote {leituras_dia + 1}/6)")
        if URL_PLANILHA != '':
            try:
                dados_enviar = {
                    "tipo": "leitura",
                    "temperatura": temperatura,
                    "umidade": umidade,
                    "gas": nivel_gas,
                    "aviso_seguranca": aviso_seguranca
                }
                res_sheets = urequests.post(URL_PLANILHA,
json=dados_enviar)
                res_sheets.close()
                print("[ PLANILHA ] -> Sincronização OK!")
            except Exception as e:
                print("[ PLANILHA ] -> Erro na sincronização:", e)

        soma_temperatura_dia += temperatura
        soma_umidade_dia += umidade
        leituras_dia += 1

        if leituras_dia >= 6:
            media_temp_calculada = soma_temperatura_dia / leituras_dia
            media_umid_calculada = soma_umidade_dia / leituras_dia
            enviar_email_relatorio(media_temp_calculada,
media_umid_calculada, leituras_dia)

        soma_temperatura_dia = 0.0
        soma_umidade_dia = 0.0
        leituras_dia = 0

# --- RESUMO NO TERMINAL E ENVIO PARA O NODE-RED ---
if time.time_diff(agora, ultimo_aviso_enviado) >= INTERVALO_AVISOS:
    if modo_ar_auto:
        modo_ar = "AUTO"
    elif ar_desligado_manual:
        modo_ar = "DESLIGADO MANUAL"
    else:
        modo_ar = "MANUAL ({}°C)".format(temperatura_alvo)

    estado_ar = "LIGADO" if rele_ar.value() == 1 else "DESLIGADO"
    estado_luz = "LIGADA" if rele_luz.value() == 1 else "DESLIGADA"

```

```

        estado_cortina = "FECHADA" if rele_cortina.value() == 1 else
"ABERTA"

        estado_alarme_texto = "ATIVADO" if alarme_ligado else "DESATIVADO"

        print("\n--- RESUMO DA CASA ---")
        print(f"Clima: {temperatura}°C | Umidade: {umidade}% | Gás:
{nivel_gas}")
        print(f"Ar Condicionado: {estado_ar} [{modo_ar}]")
        print(f"Luz: {estado_luz} | Cortina: {estado_cortina} | Modo:
{modo_atual}")
        print(f"Segurança: {estado_alarme_texto} | Alerta:
{aviso_seguranca}")
        print("-----")

        pacote_de_dados = {
            "temperatura_atual": temperatura,
            "nivel_gas": nivel_gas,
            "movimento": movimento,
            "alarme_armado": alarme_ligado,
            "ar_ligado": rele_ar.value(),
            "janela_aberta": rele_janela.value(),
            "modo_ar_auto": modo_ar_auto,
            "ar_desligado_manual": ar_desligado_manual,
            "temperatura_alvo": temperatura_alvo,
            "aviso_seguranca": aviso_seguranca,
            "modo_atual": modo_atual
        }
        mensagem_texto = json.dumps(pacote_de_dados)
        # umqtt.simple.MQTTClient.publish(topic, msg) expects the message
        # as a positional argument, not a keyword. Send the JSON string.
        messageiro.publish(CANAL_ENVIAR_DADOS, mensagem_texto)
        ultimo_aviso_enviado = agora

    except Exception as erro:
        print("Tropeço geral no sistema:", erro)

    time.sleep(0.2)

```

Figura 1 – código do Wokwi

Fonte: Autoria própria (2026).

5.2 Codigo Wokwi (upload_manual.py):

```
import sys
import time
from pathlib import Path
from mpremote.main import State, do_connect

class Args:
    def __init__(self, device):
        self.device = [device]

def _flush_input(serial):
    serial.write(b"\r\n")
    time.sleep(0.2)
    while serial.inWaiting() > 0:
        dumped = serial.read(serial.inWaiting())

def _wait_for_prompt(serial, prompt, timeout=10):
    data = b""
    start = time.monotonic()
    while time.monotonic() - start < timeout:
        if serial.inWaiting() > 0:
            data += serial.read(1)
            if data.endswith(prompt):
                return True, data
        else:
            time.sleep(0.01)
    return False, data

def enter_raw_repl(serial, max_attempts=5, timeout=10):
    prompt = b"raw REPL; CTRL-B to exit\r\n>"
    for attempt in range(1, max_attempts + 1):
        print(f'Tentando entrar no modo de gravacao ({attempt}/{max_attempts})...')
        _flush_input(serial)
        serial.write(b"\r\x03")
        time.sleep(0.05)
        while serial.inWaiting() > 0:
            serial.read(serial.inWaiting())
        serial.write(b"\r\x01")
        ok, data = _wait_for_prompt(serial, prompt, timeout=timeout)
        if ok:
            print('Modo de gravacao ativado com sucesso!')
            return True
        time.sleep(0.2)
    return False
```

```

def exec_raw_repl(serial, command, timeout=10):
    serial.write(command.encode('utf-8'))
    serial.write(b"\x04")

    # Read acknowledgment: some devices may emit prompt characters before
    # the 'OK' response (e.g. b">OK") so read bytes until we find 'OK'.
    ack = b""
    start = time.monotonic()
    while time.monotonic() - start < timeout:
        if serial.inWaiting() > 0:
            ack += serial.read(1)
            if b"OK" in ack:
                break
        else:
            time.sleep(0.01)

    if b"OK" not in ack:
        raise RuntimeError(f"Falha no comando. Resposta da placa={ack!r}")

    # Now collect output until the raw REPL EOF (0x04) is received.
    out = b""
    start = time.monotonic()
    while time.monotonic() - start < timeout:
        if serial.inWaiting() > 0:
            b = serial.read(1)
            if b == b"\x04":
                return out
            out += b
        else:
            time.sleep(0.01)
    raise RuntimeError('Tempo esgotado esperando a placa responder.')

def write_file_raw_repl(serial, dest, data, chunk_size=256):
    # O PULO DO GATO ESTÁ AQUI: Enviando em pacotes pequenos para não engasgar
    exec_raw_repl(serial, f"f = open({dest!r}, 'wb')\n")

    total_bytes = len(data)
    enviado = 0

    for i in range(0, total_bytes, chunk_size):
        chunk = data[i : i + chunk_size]
        exec_raw_repl(serial, f"f.write({chunk!r})\n")
        enviado += len(chunk)
        # Mostra o progresso no terminal pra gente não ficar no escuro
        print(f" -> Gravando {dest}: {enviado}/{total_bytes} bytes...")

```

```

exec_raw_repl(serial, "f.close()\n")

def upload_files(files_to_upload, device_uri="rfc2217://localhost:4000"):
    state = State()
    do_connect(state, Args(device_uri))
    transport = state.transport
    if not transport or not getattr(transport, 'serial', None):
        raise RuntimeError('Linha de comunicação indisponível.')

    try:
        serial = transport.serial
        print('Conectado na placa via', device_uri)

        if hasattr(transport, 'enter_raw_repl'):
            try:
                transport.enter_raw_repl()
                print('Modo de gravação ativado com sucesso!')
            except Exception as er:
                print('Falha no raw REPL nativo do mpremote:', er)
                print('Tentando fallback manual...')
                if not enter_raw_repl(serial):
                    raise RuntimeError('Nao foi possivel dominar a placa para
gravacao.') from er
                transport.in_raw_repl = True
            else:
                if not enter_raw_repl(serial):
                    raise RuntimeError('Nao foi possivel dominar a placa para
gravacao.')
                transport.in_raw_repl = True

        for filename in files_to_upload:
            local_path = Path(filename)
            if not local_path.exists():
                print(f"Aviso: Arquivo {filename} nao encontrado. Pulando...")
                continue

            data = local_path.read_bytes()
            print(f'\nIniciando upload de {filename}...')
            write_file_raw_repl(serial, filename, data)
            print(f'Upload de {filename} concluido!')

        print('\nQuase lá! Reiniciando o cérebro da placa...')
        exec_raw_repl(serial, 'import machine\nmachine.reset()')
    finally:
        if getattr(transport, 'in_raw_repl', False):
            try:

```

```

        transport.exit_raw_repl()
    except Exception:
        pass
if getattr(transport, 'serial', None):
    try:
        transport.serial.close()
    except Exception:
        pass

def parse_cli_args():
    if len(sys.argv) > 1:
        maybe_uri = sys.argv[-1]
        if maybe_uri.startswith(('rfc2217://', 'serial://', 'COM', '/dev/'))
or '://' in maybe_uri:
            arquivos = sys.argv[1:-1] or ['ssd1306.py', 'main.py']
            return arquivos, maybe_uri
        return sys.argv[1:], 'rfc2217://localhost:4000'
    return ['ssd1306.py', 'main.py'], 'rfc2217://localhost:4000'

if __name__ == '__main__':
    arquivos_para_enviar, uri = parse_cli_args()

    print("Ligando o trator de carga pesada...")
    upload_files(arquivos_para_enviar, uri)

```

Figura 2 – código do Wokwi

Fonte: Autoria própria (2026).

5.3 Planilha com data hora, temperatura, umidade:

A1	23/05/2026 17:15:44							
	A	B	C	D	E	F	G	
1	23/05/2026 17:1	21.4	40	990	Nenhum Alerta			
2	23/05/2026 17:1	21.4	40	990	Nenhum Alerta			
3	23/05/2026 17:1	21.4	40	990	Nenhum Alerta			
4	23/05/2026 17:1	21.4	40	990	Nenhum Alerta			
5	23/05/2026 17:1	21.4	40	990	Nenhum Alerta			
6	23/05/2026 17:1	21.4	40	990	Nenhum Alerta			
7	23/05/2026 17:1	21.4	40	990	Nenhum Alerta			
8	23/05/2026 17:1	21.4	40	990	Nenhum Alerta			
9	23/05/2026 17:1	21.4	40	990	Invasor Detectado no Ambiente!			
10	23/05/2026 17:1	21.4	40	990	Nenhum Alerta			
11	23/05/2026 17:1	21.4	40	990	Nenhum Alerta			
12	23/05/2026 17:1	21.4	40	990	Nenhum Alerta			
13	23/05/2026 17:1	21.4	40	2439	Alto Nivel de Gas Detectado!			
14	23/05/2026 17:2	21.4	40	2439	Alto Nivel de Gas Detectado!			
15	23/05/2026 17:2	21.4	40	962	Nenhum Alerta			
16	23/05/2026 17:2	21.4	40	962	Nenhum Alerta			
17	23/05/2026 17:2	21.4	40	962	Nenhum Alerta			
18	23/05/2026 17:2	21.4	40	962	Nenhum Alerta			
19	23/05/2026 17:2	21.4	40	962	Nenhum Alerta			
20	23/05/2026 17:2	21.4	40	962	Nenhum Alerta			
21	23/05/2026 17:2	21.4	40	962	Nenhum Alerta			
22	23/05/2026 17:2	21.4	40	962	Nenhum Alerta			
23	23/05/2026 17:2	21.4	40	962	Nenhum Alerta			
24	23/05/2026 17:2	21.4	40	962	Nenhum Alerta			
25	23/05/2026 17:2	21.4	40	962	Nenhum Alerta			
26	23/05/2026 17:2	21.4	40	962	Nenhum Alerta			
27								
28								

Figura 3 – Tabela de informações

Fonte: Autoria própria (2026).

5.3 Node-red (fluxo):

5.4 Node-red (Dashboard):

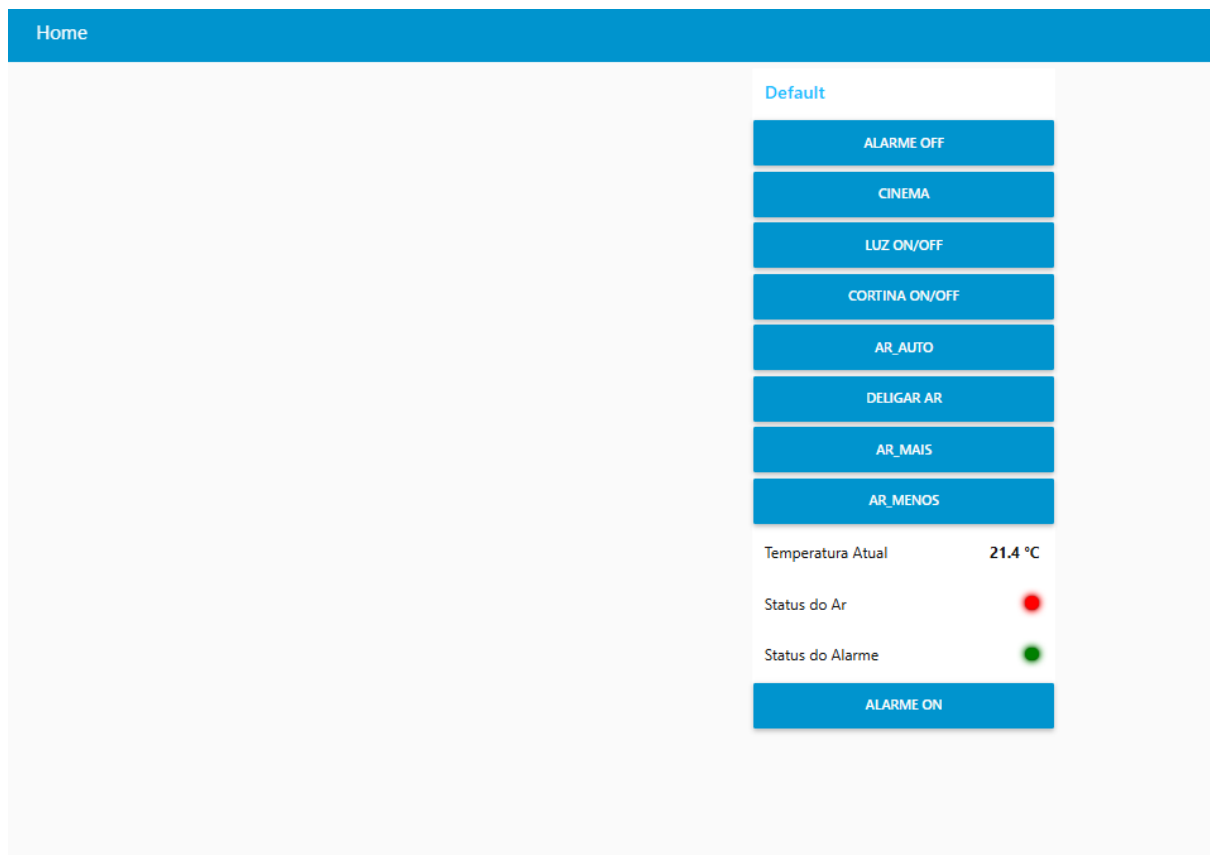


Figura 5 – Desing do modelo Node-Red (dashboard)

Fonte: Autoria própria (2026).

5.5 Node-red (Dashboard - localhost):

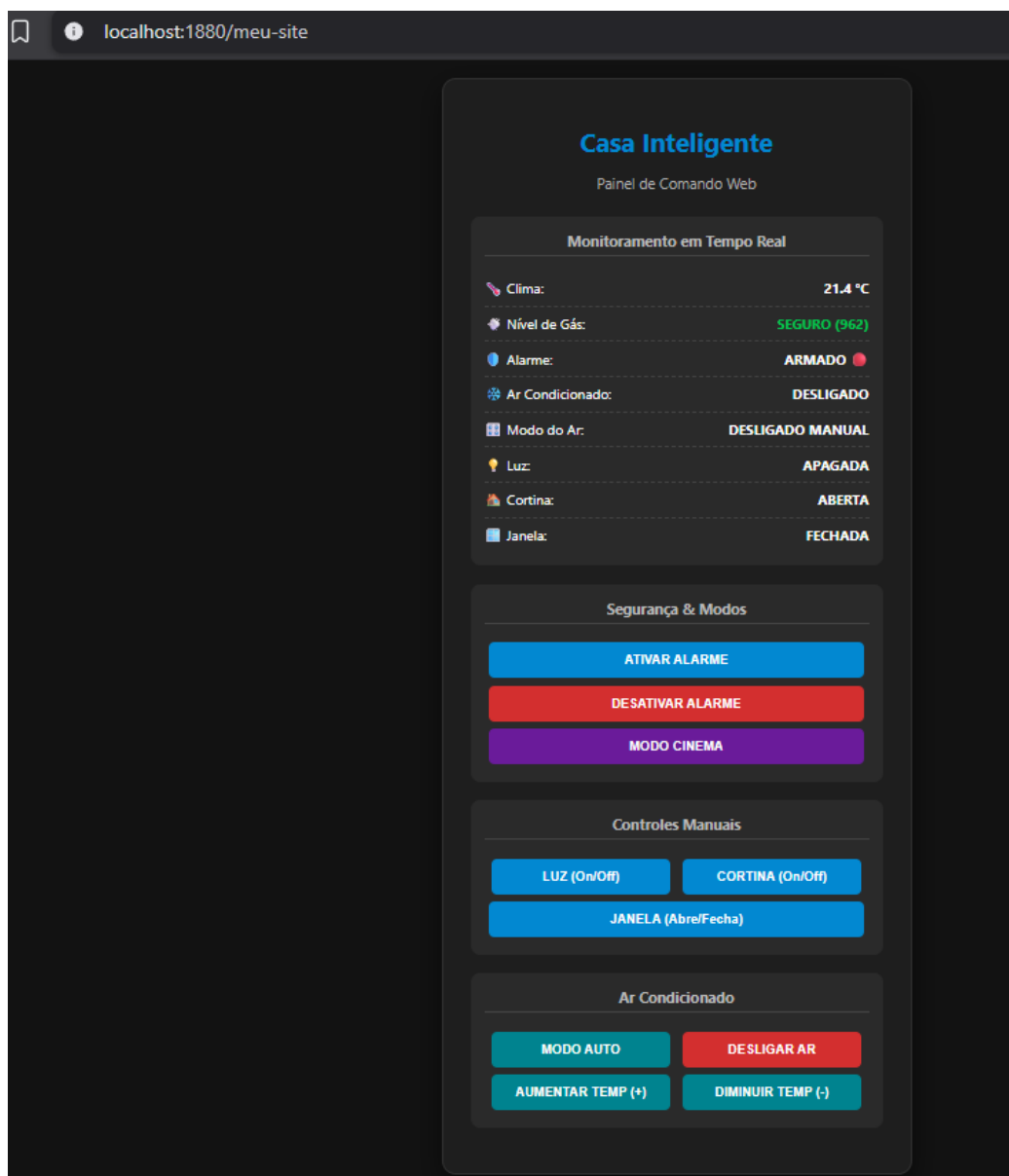


Figura 6 – Desing do modelo Node-Red (LocalHost)

Fonte: Autoria própria (2026).

5.6 Wokwi (Desing):

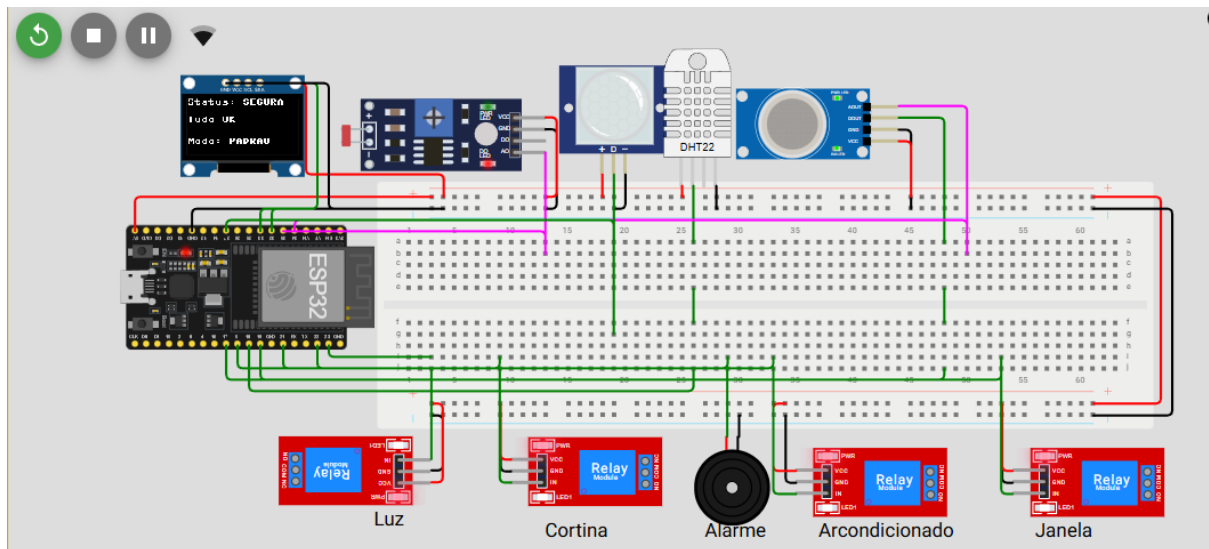


Figura 7 – Desing do modelo Wokwi (LocalHost)

Fonte: Autoria própria (2026).

5.7 HaiveMQ:

**HIVEMQ**

Websockets Client Showcase

**HIVEMQ** CLOUD

Need a fully managed MQTT broker?
Get your own Cloud broker and connect up to 100 devices for free.

Get your free account

Connection

● connected

Publish

Topic: testtopic/1

QoS: 0

Retain: ☐

Publish

Message

Subscriptions

Add New Topic Subscription

Qos: 0

gabriel/casa/sensores

Messages

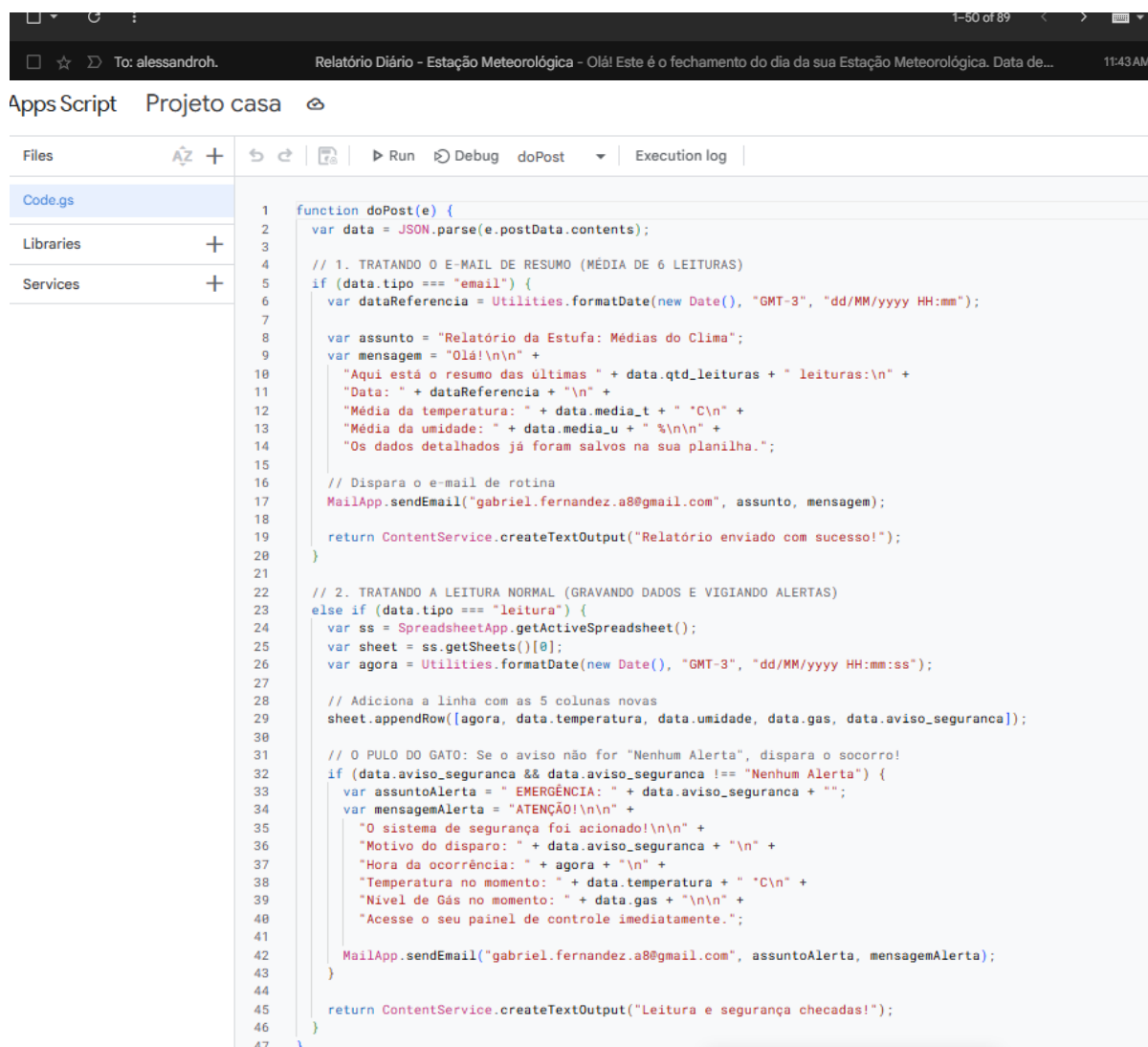
2026-05-23 18:14:59 Topic: gabriel/casa/sensores Qos: 0

```
{ "temperatura_atual": 21.4, "movimento": 0, "alarme_armado": false, "ar_ligado": 0, "modo_ar_auto": true, "temperatura_alvo": 22, "ar_desligado_manual": false, "aviso_seguranca": "Nenhum Alerta", "modo_atual": "PADRAO", "janela_aberta": 0, "nivel_gas": 1034 }
```

Figura 7 – Exibição do mensagens e conexão com HiveMQ

Fonte: Autoria própria (2026).

5.8 Apps Script para o envio de Email:



```
1 function doPost(e) {
2   var data = JSON.parse(e.postData.contents);
3
4   // 1. TRATANDO O E-MAIL DE RESUMO (MÉDIA DE 6 LEITURAS)
5   if (data.tipo === "email") {
6     var dataReferencia = Utilities.formatDate(new Date(), "GMT-3", "dd/MM/yyyy HH:mm");
7
8     var assunto = "Relatório da Estufa: Médias do Clima";
9     var mensagem = "Olá!\n\n" +
10       "Aqui está o resumo das últimas " + data.qtd_leituras + " leituras:\n" +
11       "Data: " + dataReferencia + "\n" +
12       "Média da temperatura: " + data.media_t + " °C\n" +
13       "Média da umidade: " + data.media_u + " %\n\n" +
14       "Os dados detalhados já foram salvos na sua planilha.";
15
16     // Dispara o e-mail de rotina
17     MailApp.sendEmail("gabriel.fernandez.a8@gmail.com", assunto, mensagem);
18
19     return ContentService.createTextOutput("Relatório enviado com sucesso!");
20   }
21
22   // 2. TRATANDO A LEITURA NORMAL (GRAVANDO DADOS E VIGIANDO ALERTAS)
23   else if (data.tipo === "leitura") {
24     var ss = SpreadsheetApp.getActiveSpreadsheet();
25     var sheet = ss.getSheets()[0];
26     var agora = Utilities.formatDate(new Date(), "GMT-3", "dd/MM/yyyy HH:mm:ss");
27
28     // Adiciona a linha com as 5 colunas novas
29     sheet.appendRow([agora, data.temperatura, data.umidade, data.gas, data.aviso_seguranca]);
30
31     // O PULO DO GATO: Se o aviso não for "Nenhum Alerta", dispara o socorro!
32     if (data.aviso_seguranca && data.aviso_seguranca !== "Nenhum Alerta") {
33       var assuntoAlerta = "EMERGÊNCIA: " + data.aviso_seguranca + "";
34       var mensagemAlerta = "ATENÇÃO!\n\n" +
35         "O sistema de segurança foi acionado!\n\n" +
36         "Motivo do disparo: " + data.aviso_seguranca + "\n" +
37         "Hora da ocorrência: " + agora + "\n" +
38         "Temperatura no momento: " + data.temperatura + " °C\n" +
39         "Nível de Gás no momento: " + data.gas + "\n\n" +
40         "Acesse o seu painel de controle imediatamente.";
41
42       MailApp.sendEmail("gabriel.fernandez.a8@gmail.com", assuntoAlerta, mensagemAlerta);
43     }
44
45     return ContentService.createTextOutput("Leitura e segurança checadas!");
46   }
47 }
```

Figura 8 – Exibição do Código do app Script para envio de Email

Fonte: Autoria própria (2026).

5.9 Email simulado nos testes para envio de dados diários:

Relatório da Estufa: Médias do Clima Inbox x



gabriel.fernandez.a8@gmail.com

to me ▼

Olá!

Aqui está o resumo das últimas 6 leituras:

Data: 23/05/2026 17:24

Média da temperatura: 21.4 °C

Média da umidade: 40.0 %

Os dados detalhados já foram salvos na sua planilha.

EMERGÊNCIA: Alto Nível de Gas Detectado!



gabriel.fernandez.a8@gmail.com

to me ▼

ATENÇÃO!

O sistema de segurança foi acionado!

Motivo do disparo: Alto Nível de Gas Detectado!

Hora da ocorrência: 23/05/2026 17:20:01

Temperatura no momento: 21.4 °C

Nível de Gás no momento: 2439

Acesse o seu painel de controle imediatamente.

EMERGÊNCIA: Invasor Detectado no Ambiente!



gabriel.fernandez.a8@gmail.com

to me ▼

ATENÇÃO!

O sistema de segurança foi acionado!

Motivo do disparo: Invasor Detectado no Ambiente!

Hora da ocorrência: 23/05/2026 17:18:37

Temperatura no momento: 21.4 °C

Nível de Gás no momento: 990

Acesse o seu painel de controle imediatamente.

Figura 9 – Exibição resultado do envio de Email em seu modelo Teste

Fonte: Autoria própria (2026).